

"Proyecto StreamFlix"

Contexto del Negocio:

Te acabas de incorporar al equipo de Data Engineering de StreamFlix, una plataforma de streaming ficticia. Tu objetivo es procesar los logs brutos de reproducción, cruzarlos con el catálogo de películas y generar reportes de calidad de servicio (QoS) y facturación.

Configuración del Entorno (Prerrequisito)

Antes de empezar, el alumno debe configurar su "Estación de Trabajo".

1. **IDE:** IntelliJ IDEA Community Edition (con plugin de Scala).
2. **Java:** JDK 8 o 11 (Spark tiene mejor compatibilidad histórica aquí).
3. **Build Tool:** SBT (Simple Build Tool).
4. **Estructura del Proyecto:**

Plaintext

streamflix-analytics/

|—— build.sbt

|—— src/

| |—— main/

| | |—— scala/

| | | |—— com/streamflix/

| | | |—— Main.scala <-- Punto de entrada

| | | |—— processor/ <-- Lógica de negocio

| | | |—— model/ <-- Case classes

| | |—— resources/

| | |—— data/ <-- Aquí irán los CSV/JSON

| | |—— log4j.properties <-- Para silenciar logs verbosos

Snippet build.sbt:

Scala

```
name := "StreamFlixAnalytics"
```

```
version := "0.1"
```

```
scalaVersion := "2.12.15" // O 2.13 según la versión de Spark
```

```
libraryDependencies += Seq(  
  "org.apache.spark" %% "spark-core" % "3.3.0",  
  "org.apache.spark" %% "spark-sql" % "3.3.0"  
)
```

Módulo 1: "The Raw Parse" - RDDs y Datos No Estructurados

Objetivo: Comprender la manipulación de bajo nivel con RDDs para datos sucios que un DataFrame no puede ingerir directamente. Evitar la "caja negra" de SQL al principio.

Tarea:

Recibes un archivo de logs de servidores antiguos (server_logs.txt) con un formato inconsistente. Debes parsearlo, limpiar líneas corruptas y extraer métricas básicas de errores.

Dataset (src/main/resources/data/server_logs.txt):

Plaintext

```
[INFO] 2023-10-01 10:00:01|User:102|Play:Movie_A|Dur:120  
[ERROR] 2023-10-01 10:05:00|User:999|Code:503|Msg:Service Unavailable  
WARN >> 2023-10-01 10:06:xx >> Disk usage high  
[INFO] 2023-10-01 10:10:05|User:105|Play:Movie_B|Dur:??
```

Código Guía (Main.scala):

Scala

```
// TODO: Inicializar SparkSession y obtener el SparkContext (sc)  
val rawLogsRDD = sc.textFile("src/main/resources/data/server_logs.txt")  
  
// Tarea 1: Filtrar solo las líneas que empiezan con [ERROR] o [INFO]  
// Tarea 2: Mapear para extraer (Nivel, Mensaje)  
// Tarea 3: Contar cuántos errores de tipo 503 ocurrieron usando RDD actions (count, filter)  
// Tarea 4: Calcular el porcentaje de errores respecto al total de logs válidos.
```

```
// Pregunta Reflexiva (Anti-IA):
```

```
// ¿Por qué usarías `reduceByKey` en lugar de `groupByKey` si tuviéramos millones de líneas de logs?
```

```
// ¿Qué ocurriría en memoria si usamos groupByKey con 200M de registros donde una clave concentra el 80% de los datos?
```

```
// Justifica tu respuesta con el concepto de "Shuffling".
```

Validación Manual:

- El alumno debe imprimir en consola el número exacto de líneas descartadas (corruptas).
- Debe generar un archivo de texto simple output/error_counts que contenga pares (Código, Cantidad).

Módulo 2: Estandarización con DataFrames y Schemas

Objetivo: Transición a DataFrames. Aprender a forzar tipos de datos estrictos (Schema Enforcement) y manejar valores nulos (null vs NaN).

Tarea:

Cargar un CSV de catálogo de películas (movies_metadata.csv). El archivo tiene precios en formato string con símbolos de moneda (ej: "\$12.99") y géneros separados por pipes |. Debes limpiarlo y tiparlo correctamente.

Dataset (movies_metadata.csv):

Fragmento de código

```
id,title,genres,subscription_price,release_date
101,Inception,Action|Sci-Fi,$14.99,2010-07-16
102,Toy Story,, $9.99,1995-11-22
103,Broken Record,Drama,null,invalid-date
```

Código Guía:

Scala

```
import org.apache.spark.sql.types._
import org.apache.spark.sql.functions._
```

```
// Definir Schema Manualmente (No usar inferSchema = true)
val customSchema = StructType(Array(
  StructField("id", LongType, nullable = false),
  StructField("title", StringType, nullable = false),
  // Completar el resto...
))

val moviesDF = spark.read
  .option("header", "true")
  .schema(customSchema) // ¿Qué pasa con las filas que no cumplen el schema?
  .option("mode", "DROPMALFORMED") // Experimentar con PERMISSIVE
  .csv("src/main/resources/data/movies_metadata.csv")

// TODO: Crear una UDF o usar expresiones select para limpiar el precio (quitar '$' y castear a Double)
// TODO: Hacer un análisis de nulos y duplicados en las columnas
// TODO: Manejar nulos en 'genres' reemplazando por "Unknown"
```

Validación Manual:

- **Pregunta:** "Al definir el campo subscription_price, ¿por qué en un entorno bancario/facturación real deberías usar DecimalType en lugar de DoubleType? Muestra un ejemplo de código donde DoubleType pierda precisión en una suma."
- El alumno debe mostrar el esquema final (printSchema()) y 5 filas limpias.

Módulo 3: Enriquecimiento de Datos (Joins & Broadcast)

Objetivo: Unir dos fuentes de datos (Logs de visualización + Metadatos). Comprender estrategias de Join y optimización básica.

Tarea:

Unir los logs de reproducciones (limpios del Módulo 1, convertidos a DF) con el catálogo de películas. El objetivo es saber qué Género es el más visto, no qué película.

Escenario: La tabla de movies es pequeña (100 MB), la tabla de logs es gigante (simulada 10 GB).

Código Guía:

Scala

```
// logsDF: DataFrame con columnas [user_id, movie_id, duration_watched]
// moviesDF: DataFrame con columnas [id, title, genres]

// Tarea: Realizar el Join
// IMPORTANTE: Forzar un Broadcast Join explícito para la tabla de movies
val enrichedDF = logsDF.join(
  broadcast(moviesDF),
  logsDF("movie_id") === moviesDF("id"),
  "inner"
)

// Tarea: Explotar la columna 'genres' (si una peli es Action|Sci-Fi, cuenta para ambos)
val genreMetricsDF = enrichedDF
  .withColumn("genre", explode(split(col("genres"), "\\|")))
  .groupBy("genre")
  .agg(sum("duration_watched").alias("total_hours"))
```

Validación Manual:

- El alumno debe explicar el "Physical Plan" (enrichedDF.explain()) y señalar dónde aparece BroadcastHashJoin.
- ¿Qué ocurriría si el 60% de los logs pertenecen a la misma película?
- Resultado esperado: Una tabla ordenada por horas totales de visualización.

🔗 Módulo 4: Análisis de Comportamiento (Window Functions)

Objetivo: Responder preguntas complejas que dependen del orden de los datos o agrupaciones relativas, sin usar Joins complejos.

Tarea:

Identificar sesiones de "Binge Watching" (Maratón).

Definición de Maratón: Un usuario ve más de 3 episodios/películas consecutivas con menos de 20 minutos de pausa entre ellas.

Dataset: Logs limpios con timestamp.

Código Guía:

Scala

```
import org.apache.spark.sql.expressions.Window

val windowSpec = Window.partitionBy("user_id").orderBy("timestamp")

val lagDF = logsDF.withColumn("prev_timestamp", lag("timestamp", 1).over(windowSpec))

// TODO: Calcular la diferencia en minutos entre 'timestamp' y 'prev_timestamp'
// TODO: Crear una columna "is_binge" si la diferencia < 20 mins
// TODO: Generar un reporte de los "Top 10 Binge Watchers"
```

Validación Manual:

- **Análisis:** El dataset contiene un usuario User:999 que tiene tiempos negativos (errores de reloj). El alumno debe identificarlo y explicar cómo su lógica manejó ese caso (¿lo filtró antes o rompió el cálculo?).
- **Entregable:** ID de los 3 usuarios más adictos.

Módulo 5: Particionamiento y Escritura Eficiente

Objetivo: Entender que escribir datos es tan importante como leerlos. Formatos columnares (Parquet) y particionamiento.

Tarea:

El equipo de Data Science necesita los datos procesados para entrenar modelos, pero sus consultas son siempre filtrando por fecha y país.

Requerimientos:

1. Escribir el resultado final en formato **Parquet**.
2. Particionar los datos por year y country.
3. Modo de escritura: Overwrite.

Código Guía:

Scala

```
// TODO: Asegurarse de que las columnas de partición existan y estén limpias
finalReportDF.write
  .mode(SaveMode.Overwrite)
```

```
.partitionBy("year", "country")  
.parquet("src/main/resources/output/analytics_warehouse")
```

Validación Manual:

- El alumno debe mostrar una captura de pantalla de su explorador de archivos mostrando la estructura de carpetas creada:

Plaintext

/output/analytics_warehouse/year=2023/country=ES/part-0000.parquet

/output/analytics_warehouse/year=2023/country=US/part-0000.parquet

- Pregunta: "¿Por qué NO debemos particionar por user_id en este caso? (Pista: Small files problem)".

Módulo 6: Proyecto Integrador (El ETL Final)

Objetivo: Unir todo en un flujo coherente simulando un job productivo.

Escenario:

Crear una aplicación (Main.scala) que acepte argumentos de entrada (input path, output path).

El Job debe:

1. Leer logs (Raw).
2. Limpiar y Tipar.
3. Enriquecer con Metadata.
4. Calcular KPIs (Top géneros y Binge Watchers).
5. Guardar resultados en Parquet particionado.

Requisito Profesional:

El código debe incluir manejo de excepciones Try/Catch para la lectura de archivos y usar un logger (Log4j) para informar el progreso, no println.

Entregable Final:

- Código fuente completo en .zip.
- Un archivo README.md explicando cómo ejecutar el JAR generado con spark-submit.
- Un breve reporte (PDF) interpretando los datos: "¿Qué género tuvo mayor retención en 2023 según tus resultados?".